Doubly Circular Linked List

1.Insertion at beginning

main.c

Share Run

Output

```c
#include <stdio.h>
#include <stdlib.h>

struct node{
    int data;
    struct node *prev;
    struct node *next;
};

struct node *head = NULL;

void beginning() {
    struct node * first,*second,*tail;

    first = (struct node*)malloc(sizeof(struct node));
    second = (struct node*)malloc(sizeof(struct node));
    tail = (struct node*)malloc(sizeof(struct node));

    first->data = 20;
    second->data = 30;
    tail->data = 40;

    head =  first;

     first->next = second;
    second->prev =  first;

    second->next =tail;
tail->prev = second;

    tail->next = head;
    head->prev = tail;
}

void insertbegin(int data){
    struct node *newnode,*tail;

    newnode = (struct node*)malloc(sizeof(struct node));
    newnode->data = data;

   tail = head->prev;

    newnode->next = head;
    newnode->prev =tail;

    tail->next = newnode;
    head->prev = newnode;

    head = newnode;
}

void display(){
    struct node *temp = head;
```

```
List: 10 20 30 40

=== Code Execution Successful ===
```

```c
}

void display(){
    struct node *temp = head;

    if(head == NULL){
        printf("List is empty");
        return;
    }

    printf("%d ", temp->data);
    temp = temp->next;

    while(temp != head){
        printf("%d ", temp->data);
        temp = temp->next;
    }
}

int main(){
    beginning();
    insertbegin(10);

    printf("List: ");
    display();

    return 0;
}
```

```
List: 10 20 30 40

=== Code Execution Successful ===
```

2. Insertion at End

main.c

Share Run

```c
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *prev, *next;
};

struct node *head = NULL;

void display() {
    struct node *temp = head;
    while(temp != NULL) {
        printf("%d <-> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

void insertatEnd(int value) {
    struct node *newnode, *temp;

    newnode = (struct node*)malloc(sizeof(struct node));
    newnode->data = value;
    newnode->next = NULL;

    if(head == NULL) {
        newnode->prev = NULL;
        head = newnode;
        return;
    }

    temp = head;
    while(temp->next != NULL) {
        temp = temp->next;
    }

    temp->next = newnode;
    newnode->prev = temp;
}

int main() {

    insertatEnd(10);
    insertatEnd(20);

    printf("Before insertion:  ");
    display();

    insertatEnd(30);
    printf("After insertion:  ");
    display();
    return 0;
}
```

Output

```
Before insertion:  10 <-> 20 <-> NULL
After insertion:  10 <-> 20 <-> 30 <-> NULL


=== Code Execution Successful ===
```

3.Insertion at Position.

```c
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *prev, *next;
};

struct node *head = NULL;

void insertatEnd(int value) {
    struct node *newnode, *temp;

    newnode = (struct node*)malloc(sizeof(struct node));
    newnode->data = value;
    newnode->next = NULL;

    if(head == NULL) {
        newnode->prev = NULL;
        head = newnode;
        return;
    }

    temp = head;
    while(temp->next != NULL)
        temp = temp->next;

    temp->next = newnode;
    newnode->prev = temp;
}

void insertatPosition(int value, int pos) {
    struct node *newnode, *temp;
    int i;

    newnode = (struct node*)malloc(sizeof(struct node));
    newnode->data = value;

    temp = head;

    for(i = 1; i < pos-1; i++) {
        temp = temp->next;
    }

    newnode->next = temp->next;
    newnode->prev = temp;

    if(temp->next != NULL)
        temp->next->prev = newnode;

    temp->next = newnode;
}

void display() {
```

```
10 <-> 20 <-> 40 <-> NULL
10 <-> 20 <-> 30 <-> 40 <-> NULL


=== Code Execution Successful ===
```

```c
void display() {
    struct node *temp = head;
    while(temp != NULL) {
        printf("%d <-> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

int main() {

    insertatEnd(10);
    insertatEnd(20);
    insertatEnd(40);

    display();
    insertatPosition(30, 3);
    display();
    return 0;
}
```

```
10 <-> 20 <-> 40 <-> NULL
10 <-> 20 <-> 30 <-> 40 <-> NULL


=== Code Execution Successful ===
```

4. Deletion at Beginning.

main.c

Share Run

```c
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *prev, *next;
};

struct node *head = NULL;

void insertatEnd(int value) {
    struct node *newnode, *temp;

    newnode = (struct node*)malloc(sizeof(struct node));
    newnode->data = value;
    newnode->next = NULL;

    if(head == NULL) {
        newnode->prev = NULL;
        head = newnode;
        return;
    }

    temp = head;
    while(temp->next != NULL) {
        temp = temp->next;
    }
    temp->next = newnode;
    newnode->prev = temp;
}

void deleteatBeginning() {
    struct node *temp;

    if(head == NULL) {
        printf("List is empty\n");
        return;
    }

    temp = head;
    head = head->next;

    if(head != NULL)
        head->prev = NULL;

    free(temp);
}

void display() {
    struct node *temp = head;
    while(temp != NULL) {
        printf("%d <-> ", temp->data);
        temp = temp->next;
    }
```

Output

```
Before deletion:  100 <-> 200 <-> 300 <-> NULL
After deletion:  200 <-> 300 <-> NULL


=== Code Execution Successful ===
```


```
Before deletion:  100 <-> 200 <-> 300 <-> NULL
After deletion:  200 <-> 300 <-> NULL


=== Code Execution Successful ===
```

```
41      temp = head;
42      head = head->next;
43
44      if(head != NULL)
45          head->prev = NULL;
46
47      free(temp);
48  }
49
50  void display() {
51      struct node *temp = head;
52      while(temp != NULL) {
53          printf("%d <-> ", temp->data);
54          temp = temp->next;
55      }
56      printf("NULL\n");
57  }
58  int main() {
59      insertatEnd(100);
60      insertatEnd(200);
61      insertatEnd(300);
62      printf("Before deletion:  ");
63      display();
64      deleteatBeginning();
65      printf("After deletion:  ");
66      display();
67      return 0;
68  }
```

```
Before deletion:  100 <-> 200 <-> 300 <-> NULL
After deletion:  200 <-> 300 <-> NULL


=== Code Execution Successful ===
```

5.Deletion at End

main.c

Share Run

Output

```c
#include <stdio.h>
#include <stdlib.h>

struct node{
    int data;
    struct node *prev;
    struct node *next;
};

struct node *head = NULL;

void end(){
    struct node *first,*second;

    head = (struct node*)malloc(sizeof(struct node));
    first = (struct node*)malloc(sizeof(struct node));
    second = (struct node*)malloc(sizeof(struct node));

    head->data = 65;
    first->data = 115;
    second->data = 30;

    head->prev = NULL;
    head->next = first;

    first->prev = head;
    first->next = second;
    second->prev = first;
    second->next = NULL;
}

void delete_end(){
    struct node *temp = head;

    if(head == NULL){
        printf("List is empty\n");
        return;
    }

    while(temp->next != NULL){
        temp = temp->next;
    }

    temp->prev->next = NULL;
    free(temp);
}

void display(){
    struct node *temp = head;

    while(temp != NULL){
        printf("%d ", temp->data);
        temp = temp->next;
    }
```

```
Before deletion: 65 115 30
After deletion: 65 115

=== Code Execution Successful ===
```


```
Before deletion: 65 115 30
After deletion: 65 115

=== Code Execution Successful ===
```

```
    }

    temp->prev->next = NULL;
    free(temp);
}

void display(){
    struct node *temp = head;

    while(temp != NULL){
        printf("%d ", temp->data);
        temp = temp->next;
    }
}

int main(){
    end();

    printf("Before deletion: ");
    display();

    delete_end();

    printf("\nAfter deletion: ");
    display();

    return 0;
}
```

```
Before deletion: 65 115 30
After deletion: 65 115

=== Code Execution Successful ===
```

6.Deletion at Position.

main.c | Share | Run | Output

```
#include <stdio.h>
#include <stdlib.h>

struct node{
    int data;
    struct node *prev;
    struct node *next;
};

struct node *head = NULL;

void create(){
    struct node *first, *second;

    head = (struct node*)malloc(sizeof(struct node));
    first = (struct node*)malloc(sizeof(struct node));
    second = (struct node*)malloc(sizeof(struct node));

    head->data = 10;
    first->data = 20;
```

```
Before deletion: 10 20 30
After deletion: 10 30

=== Code Execution Successful ===
```

```
    first->data = 20;
    second->data = 30;

    head->prev = NULL;
    head->next = first;

    first->prev = head;
    first->next = second;

    second->prev = first;
    second->next = NULL;
}

void delete_pos(int pos){
    struct node *temp = head;
    int i;

    if(head == NULL){
        printf("List is empty\n");
        return;
```

```
Before deletion: 10 20 30
After deletion: 10 30

=== Code Execution Successful ===
```

```
        return;
    }

    if(pos == 1){
        head = head->next;
        if(head != NULL)
            head->prev = NULL;
        free(temp);
        return;
    }

    for(i = 1; i < pos-1; i++){
        temp = temp->next;
    }

    struct node *del = temp->next;

    temp->next = del->next;

    if(del->next != NULL)
        del->next->prev = temp;

    free(del);
}

void display(){
    struct node *temp = head;
```

```
Before deletion: 10 20 30
After deletion: 10 30

=== Code Execution Successful ===
```

```
        del->next->prev = temp;

    free(del);
}

void display(){
    struct node *temp = head;

    while(temp != NULL){
        printf("%d ", temp->data);
        temp = temp->next;
    }
}

int main(){

    create();

    printf("Before deletion: ");
    display();

    delete_pos(2);

    printf("\nAfter deletion: ");
    display();

    return 0;
}
```

```
Before deletion: 10 20 30
After deletion: 10 30

=== Code Execution Successful ===
```